

Fynd AI Intern Assessment - Project Report

Submitted by: Gaurav Thakur
Date: 16/12/2025

Links:

- **GitHub Repository:** [Github Repo Link](#)
- **Live Application:** [Live App Link \(HuggingFace\)](#)

1. Task 1: Rating Prediction via Prompting

1.1. Objective

The goal was to classify Yelp reviews into 1-5 star ratings using an LLM (Groq/Llama-3) and strictly output valid JSON. Three different prompting strategies were designed and evaluated on a subset of 200 reviews.

1.2. Prompting Approaches

- **Approach 1: Zero-Shot (Direct)**
 - **Design:** I provided the model with the review and a strict instruction to output the rating and explanation in JSON format, without providing any examples.
 - **Hypothesis:** This tests the model's innate ability to understand sentiment without guidance.
- **Approach 2: Few-Shot (With Examples)**
 - **Design:** I included 3 distinct examples (Negative, Neutral, Positive) in the prompt before the target review.
 - **Hypothesis:** Providing context helps the model calibrate its scoring, especially for neutral or mixed reviews.
- **Approach 3: Chain-of-Thought (Reasoning First)**
 - **Design:** The prompt explicitly instructed the model to "think step-by-step" (analyze keywords, assess tone) *before* assigning a final score.
 - **Hypothesis:** Forcing a reasoning step reduces impulsive hallucinations and improves accuracy on complex/sarcastic reviews.

1.3. Evaluation & Results

- **Dataset:** 200 randomly sampled rows from the Yelp Reviews dataset.
- **Model Used:** llama-3.1-8b-instant (via Groq API).

Approach	Accuracy (%)	JSON Validity (%)
Zero-Shot	60.5%	100%
Few-Shot	61.5%	100%
Chain-of-Thought	60.5%	100%

(Note: Replace [INSERT SCORE] with the numbers from your notebook output)

1.4. Analysis

- **JSON Validity:** By enforcing a system prompt ("You are a helpful assistant that outputs strictly in JSON format") and using the `response_format={"type": "json_object"}` parameter, I achieved a **100% valid JSON rate** across all experiments.
- **Performance Comparison:**
 - *Example observation (Edit based on your results):* The **Few-Shot** approach generally performed best because the examples helped anchor the model's definition of a "3-star" vs "4-star" review.
 - *Example observation:* The **Chain-of-Thought** approach was slightly slower due to longer output generation but provided much richer explanation fields.

2. Task 2: AI Feedback System (Web Application)

2.1. System Architecture

I built a unified web application using **Streamlit (Python)**, deployed on **Hugging Face Spaces**.

- **Frontend:** Streamlit offering two distinct views (User & Admin) selectable via a sidebar.
- **Backend Logic:** Python functions handle data processing and API calls to Groq.
- **Database:** A shared `reviews.csv` file serves as the single source of truth. This ensures immediate data consistency between the User and Admin dashboards.
- **AI Engine:** Integrated `llama-3.1-8b-instant` for three distinct tasks:
 1. Generating polite customer replies.
 2. Summarizing reviews into 5-word snippets.
 3. Suggesting recommended actions for the admin.

2.2. Design Decisions

1. **Unified App Structure:**

Instead of deploying two separate apps, I combined them into one Streamlit application with a sidebar navigation. This was a critical architectural choice to satisfy the requirement of a "shared data source" without needing an external cloud database (like SQL/Firebase). It ensures that as soon as a user submits a review, the Admin dashboard updates instantly.
2. **Robust Error Handling:**

The application includes fallback logic for the LLM. If the API fails or returns a malformed response (e.g., missing the "|" separator in the admin analysis), the system defaults to "Manual Review Required" rather than crashing the interface.
3. **Security:**

API keys are managed via Environment Secrets (Hugging Face) rather than hardcoding, ensuring the codebase is secure and public-ready.

2.3. System Behavior

- **User Flow:** A user selects a rating (1-5) and submits text. The system displays a loading spinner, sends the text to the LLM, saves the result, and immediately presents an empathetic AI-generated response.
- **Admin Flow:** The admin sees a dataframe sorted by "newest first." Key metrics (Total Reviews, Negative Review Count) are calculated instantly using Pandas. The "Recommended Action" column allows the admin to quickly triage issues (e.g., "Offer refund" vs "Send thank you note") without reading full paragraphs.

2.4. Limitations & Future Work

- **Data Persistence:** Currently, data is stored in a CSV file within the container. On free cloud tier restarts, this data may be wiped. For production, I would connect this to a persistent database like PostgreSQL or Google Sheets.
-

3. Conclusion

This project successfully demonstrates the integration of Large Language Models into practical software engineering workflows. By combining prompt engineering (Task 1) with full-stack development (Task 2), I created a functional, deployed end-to-end feedback loop that automates customer interaction and insight generation.